

FAANG Interview Preparation

LC 1976: Number of Ways to Arrive at Destination

1. Problem Overview & State Variables

- **Graph Properties:** n cities, numbered from 0 to $n-1$.
- **Objective:** We only need to consider *shortest time travels*. Our primary goal is finding the **MINIMUM DISTANCE**.

State Arrays

- `dist[]` → Will store the shortest distance to a node.
- `ways[]` → Will store the number of shortest paths to a node.

Initialization

- `dist[source] = 0`
- `ways[source] = 1`

2. Relaxation Logic (Core)

Case 1: Finding a Strictly Shorter Path

Suppose `curr dist = 10` and `edge = 5`.

If `new = 15` and `15 < 20` (the previous known distance):

- `dist[next] = 15;`
- `ways[next] = ways[curr];`

Reasoning: Every shortest path to 'next' comes through 'curr'.

Case 2: Finding an Equal Shortest Path

We found `15 == 15` (the new calculated distance equals the known shortest distance).

We now have multiple shortest paths converging.

- `ways[next] += ways[curr];`

| 3. C++ Implementation

```

class Solution {
public:
    int const MOD=1e9+7;
    int countPaths(int n, vector<vector<int>>& roads) {

        // step1 is to build adjacency list and fill it
        vector<vector<pair<int,int>>> adj(n);

        for(auto &it:roads){
            int u=it[0];
            int v=it[1];
            int t=it[2];

            adj[u].push_back({v,t});
            adj[v].push_back({u,t});
        }

        // step2 build totaltime matrix and number of ways and priority queue to store
        priority_queue<pair<long long,int>,vector<pair<long long,int>>,greater<pair<long long,int>>> pq;
        vector<long long> time(n,LLONG_MAX);
        vector<long long> ways(n,0);

        time[0]=0;
        ways[0]=1;

        pq.push({0,0});

        while(!pq.empty()){
            int currnode=pq.top().second;
            long long currtime=pq.top().first;

            pq.pop();
            if(currtime>time[currnode]){
                continue;
            }

            for(auto &it:adj[currnode]){
                int nextnode=it.first;
                long long cost=it.second;

                long long newtime=currtime+cost;
                if(newtime<time[nextnode]){
                    time[nextnode]=newtime;
                    ways[nextnode]=ways[currnode];
                    pq.push({newtime,nextnode});
                }
                else if(newtime==time[nextnode]){
                    ways[nextnode]+=ways[currnode]%MOD; // since dist,node is already in pq
                } // ek aur path mil gaya is liye
            }
        }
    }
};

```

```
        }  
    }  
  
    return ways[n-1]%MOD;  
}  
};
```